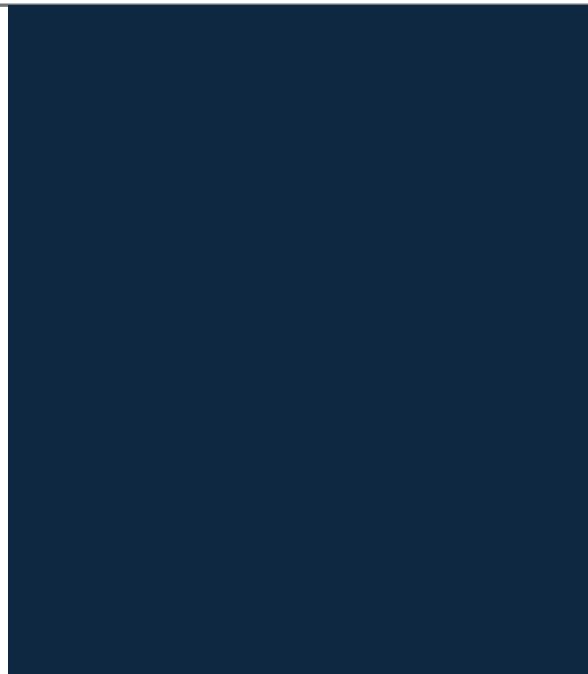




Advanced Programming

Casey Du Plessis

Casey Du Plessis



Introduction

This report documents the design, development and testing of a Data Validation System built for the Advanced Programming II module at Gauteng City College. The system is a desktop application developed in Java using the Swing framework. Its purpose is to capture a set of personal details through a graphical interface and to check every field before it is accepted, giving the user immediate and clear feedback when an entry is invalid.

The application demonstrates five categories of validation covered in the module: presence checks, type checks, length checks, format checks and range checks. Alongside the working system, this document explains the design decisions made, the validation logic used, and the testing carried out to confirm the system behaves correctly.

Building phase before

```

-      * WARNING: Do NOT modify this code. The content of this method is always
-      * regenerated by the Form Editor.
-      */
-      @SuppressWarnings("unchecked")
-      Generated Code
-
-      private void txtSubActionPerformed(java.awt.event.ActionEvent evt) {
-          // TODO add your handling code here:
-      }
-
-      private void txtNASActionPerformed(java.awt.event.ActionEvent evt) {
-          String Name= name.getText().toString();
-          String Surname= surname.getText().toString();
-      }
-
-      // Variables declaration - do not modify
-      private javax.swing.JLabel jLabel1;
-      private javax.swing.JLabel jLabel2;
-      private javax.swing.JLabel jLabel3;
```

Registration and Validation	
Name & Surname:	
ID Number/ Student Number:	
Date of Birth:	
Gender:	☐ Male ☐ Female
Phone Number:	
Email Address:	

After building

```
private void Namevalid() {
    String name = txtName.getText().trim();

    if(name.isEmpty()){
        display1.setText("Field is empty");
        display1.setForeground(Color.red);
    }
    else if(name.matches("[a-zA-Z ]+") && name.length() >= 3 && name.length()
        display1.setText("Valid");
        display1.setForeground(Color.green);
    }
    else{
        display1.setText("Enter a valid name");
        display1.setForeground(Color.red);
    }
}

private void IDvalid() {
    String Student_ID_Number = txtID.getText();

    if(!Student_ID_Number.matches("\\d+")){
        display3.setText("Enter numbers only");
        display3.setForeground(Color.red);
    }
}
```

The screenshot shows a Java Swing application window titled "Validation Form". On the left, there is a dark red sidebar containing two buttons: "Exit" and "Home". The main content area has an orange background. It features a form with the following elements:

- A label "Name & Surname:" followed by a text field.
- A label "Phone Number" followed by a text field.
- A label "Student ID Number:" followed by a text field.
- A label "Gender" followed by two radio buttons labeled "Male" and "Female".
- A label "Email" followed by a text field.
- A label "Date Of Birth:" followed by a text field.
- At the bottom, there are two buttons: "Submit" and "Clear".

The title bar of the window shows "SARS" and "Validation Form".

With the before and after there were clear differences between the two and a major overall especially with it

4.1 Name and Surname

A presence check confirms the field has not been left empty. A format check, using the regular expression `^[a-zA-Z\s'-]{2,30}$`, confirms the entry contains only letters, spaces, apostrophes and hyphens, which allows for names such as O'Brien or Van der Merwe

while rejecting numbers or symbols. A length check within the same expression requires the name to be between 2 and 30 characters, long enough to be realistic and short enough to catch obviously invalid input.

4.2 ID / Student Number

A presence check confirms the field is not empty. A type check confirms the entry contains digits only, using the pattern `\d`. A length check confirms the entry is exactly 13 characters long, matching the length of a South African ID number.

4.3 Date of Birth

A presence check confirms the field is not empty. A format check confirms the date follows the `dd/mm/yyyy` pattern, so the system can reliably interpret the value entered.

4.4 Gender

Because this field uses radio buttons rather than free text, only a presence check applies. The system confirms that one of the two options has been selected before the form is treated as valid, and a `ButtonGroup` is used so that only one option can be selected at a time.

4.5 Contact Number

A presence check confirms the field is not empty. A type check confirms the entry contains digits only. A length check and a format check together, using the pattern `^0\d{9}$`, confirm the entry is exactly 10 digits long and begins with a 0, matching the format of a local mobile number.

4.6 Email Address

A presence check confirms the field is not empty. A format check, using the pattern `^[\\w.+~]+@[\\w-]+\\.([a-zA-Z]{2,})$`, confirms the entry contains an @ symbol followed by a valid domain structure, matching the general shape of a real email address.

5. Testing

This section provides evidence that each validation check behaves correctly, first with invalid data producing an error message, and then with valid data being accepted. A final screenshot confirms the complete form passing validation on every field.

Conclusion

With this system I had to watch a lot of videos and ask for ideas on how to go and even having to search more ways to use Java since im STILL NEW but I enjoyed the challenge since it gave me excitement and now I still have my final year project to go for and IM looking forward to it but to make it proper and more classic then this one since I tried real hard but I will improve

References:

<https://youtu.be/pQ0cLUIFzv0?si=7HB6P-HHMbn6UeMT>

<https://youtu.be/d0Vz3xMYKMo?si=S3PLIRbjjeJsSQGO>

https://youtu.be/3Jr1mRRrESk?si=H_0vekae-grLykA3

<https://youtu.be/54yGDNLjzro?si=PFQovgpuCeQp5Hp6>

<https://youtu.be/tMG0eM9XETU?si=ps-AujnTEDAv8let>